

ATIVIDADE PRÁTICA 07: PADRÕES DE PROJETO (DECORATOR)

Cenário:

Imagine que você está desenvolvendo um editor de texto simples. Inicialmente, o editor permite apenas exibir o texto digitado pelo usuário. Porém, o sistema deverá permitir que novas funcionalidades de formatação sejam adicionadas, como negrito, itálico e sublinhado.

Uma possibilidade seria criar uma classe diferente para cada combinação de funcionalidades, como TextoNegrito, Textoitálico, TextoNegritoItálico, TextoNegritoItálicoSublinhado etc. À medida que novas funcionalidades fossem adicionadas, o número de classes cresceria rapidamente.

Utilize o padrão Decorator para permitir que funcionalidades de formatação sejam adicionadas dinamicamente a um objeto, sem modificar a classe original do editor.

Requisitos:

Utilize o padrão Decorator para desacoplar o editor básico das funcionalidades adicionais de formatação, permitindo combinar diferentes decoradores de forma flexível.

O sistema deve possuir, no mínimo:

- um componente abstrato que represente o editor de texto;
- um editor de texto concreto, responsável pelo comportamento básico;
- uma classe decoradora abstrata;
- pelo menos três decoradores concretos: NegritoDecorator, ItálicoDecorator e SublinhadoDecorator;
- um programa de teste que permita combinar diferentes funcionalidades.

Etapas da Atividade:

A atividade é composta por 5 etapas, entregues ao longo do período. Leia cada uma com atenção — todas são obrigatórias e avaliadas.

Etapa 1 — Modelagem UML (individual)

Antes de escrever qualquer código, elabore o diagrama de classes UML do sistema.

No diagrama, identifique e rotule os papéis do padrão Decorator:

- Component
- ConcreteComponent
- Decorator
- ConcreteDecorator

Represente também o relacionamento que permite que diferentes decoradores sejam encadeados.

Importante: nesta etapa, não é permitido escrever código. O objetivo é modelar a solução antes de implementá-la. **Use Astah ou Visual Paradigm para criar o diagrama de classes.**

Entrega: diagrama UML com os papéis do padrão identificados.

Etapla 2 — Implementação e revisão crítica (individual)

Implemente o sistema completo aplicando o padrão Decorator. Você pode utilizar ferramentas de IA como apoio, mas deve entregar, junto com o código:

O prompt exato que você utilizou na ferramenta de IA (se tiver usado);

Uma revisão crítica da resposta obtida, apontando:

- o que está correto e por quê;
- o que está incorreto ou incompleto e por quê;
- o que você modificou em relação à resposta original e a justificativa de cada alteração.

A revisão deve relacionar o código ao diagrama da Etapa 1 e aos conceitos do GoF.

Importante: o objetivo não é proibir o uso de IA, mas garantir que você compreenda e valide o que ela produz. Revisar criticamente é parte do trabalho de um engenheiro de software.

Entrega: código funcional + prompt utilizado + revisão crítica escrita.

Etapla 3 — Extensão do sistema (individual)

O editor precisa agora oferecer duas novas funcionalidades:

- **TachadoDecorator** — aplica o efeito de texto tachado;
- **CaixaAltaDecorator** — transforma o texto em letras maiúsculas.

Adicione as duas funcionalidades **sem modificar nenhuma classe ou interface existente**.

As novas funcionalidades devem poder ser combinadas com os decoradores já existentes.

Por exemplo, deve ser possível criar uma configuração equivalente a:

Editor básico → Negrito → Sublinhado → Caixa Alta

ou

Editor básico → Itálico → Tachado.

Em seguida, responda por escrito:

1. Quais classes você criou e por quê?
2. Por que o padrão Decorator permitiu adicionar essas funcionalidades sem modificar as classes existentes?
3. Como o encadeamento dos decoradores funciona?
4. Qual é a diferença entre:
 - criar um único decorador responsável por todas as formatações;
 - criar um decorador separado para cada funcionalidade?
5. O que aconteceria se o sistema tivesse sido implementado criando uma classe para cada combinação possível, como:
 - TextoNegrito;
 - Textoitálico;

- TextoNegritoItalico;
 - TextoNegritoItalicoSublinhado;
 - etc.?
6. Compare essa abordagem com a solução baseada em Decorator, considerando principalmente:
- quantidade de classes;
 - possibilidade de combinação;
 - facilidade de extensão;
 - manutenção do código.

Entrega

- código atualizado e funcional;
- novas funcionalidades implementadas;
- respostas justificadas.

Etapas 4 — Análise de uma solução alternativa (individual)

Considere que outro desenvolvedor decidiu implementar o sistema utilizando uma única classe `EditorDeTexto` contendo atributos e condicionais para controlar as formatações.

Por exemplo, o código poderia possuir uma lógica semelhante a:

- se negrito estiver ativado, aplicar negrito;
- se italico estiver ativado, aplicar itálico;
- se sublinhado estiver ativado, aplicar sublinhado;
- e assim por diante.

Análise essa solução e responda:

1. Qual é o principal problema dessa abordagem à medida que novas funcionalidades são adicionadas?
2. O que tende a acontecer com a classe `EditorDeTexto` quando novas formatações são solicitadas?
3. Essa abordagem viola algum princípio de projeto discutido em aula? Justifique.
4. Como o padrão Decorator modifica essa estrutura?
5. Em que situação uma solução mais simples, sem Decorator, poderia ser suficiente?

Entrega

Análise escrita e justificada, utilizando conceitos de orientação a objetos e padrões de projeto.